

Day 11: NumPy Fundamentals (Detailed Notes for Data Analysis Students)

What is NumPy?

NumPy stands for **Numerical Python**.

It is the most important Python library for numerical computing and forms the foundation of libraries like:

- Pandas
- Scikit-learn
- TensorFlow
- Matplotlib
- SciPy

Almost every Data Analyst and Data Scientist uses NumPy.

Why Do We Need NumPy?

Suppose you have sales data for 10 million customers.

Using normal Python lists:

- Slow
- More memory usage
- Difficult calculations

Using NumPy:

- ⚡ Extremely Fast
 - 💾 Less Memory
 - 📊 Easy Mathematical Operations
 - 📈 Perfect for Data Analysis
-

Installation

```
pip install numpy
```

Check Version

```
import numpy as np

print(np.__version__)
```

Import NumPy

```
import numpy as np
```

`np` is an alias (short name).

Instead of writing

```
numpy.array()
```

we write

```
np.array()
```

What is an Array?

An array stores multiple values of the **same data type**.

Python List

```
marks = [75,80,90,95]
```

NumPy Array

```
import numpy as np

marks = np.array([75,80,90,95])

print(marks)
```

Output

```
[75 80 90 95]
```

Difference Between List and NumPy Array

Python List	NumPy Array
Slower	Faster
More Memory	Less Memory
Mixed Data Types	Usually Same Data Type
Basic Operations	Powerful Mathematical Operations

Creating Arrays

1D ARRAY

```
import numpy as np

arr = np.array([10,20,30,40])

print(arr)
```

Output

```
[10 20 30 40]
```

2D ARRAY

```
import numpy as np

arr = np.array([
    [10,20],
```

```
        [30,40]  
    ])  
  
    print(arr)
```

Output

```
[[10 20]  
 [30 40]]
```

3D ARRAY

```
import numpy as np  
  
arr = np.array([  
    [  
        [1,2],  
        [3,4]  
    ],  
    [  
        [5,6],  
        [7,8]  
    ]  
)  
  
print(arr)
```

Array Dimensions

```
import numpy as np  
  
arr = np.array([10,20,30])  
  
print(arr.ndim)
```

Output

```
1
```

2D

```
arr = np.array([
    [1,2],
    [3,4]
])

print(arr.ndim)
```

Output

```
2
```

Array Shape

Shape tells rows and columns.

```
arr = np.array([
    [10,20],
    [30,40],
    [50,60]
])

print(arr.shape)
```

Output

```
(3,2)
```

Meaning

```
3 Rows
2 Columns
```

Array Size

```
arr = np.array([
    [1,2],
    [3,4]
])

print(arr.size)
```

Output

```
4
```

Array Data Type

```
arr = np.array([10,20,30])  
  
print(arr.dtype)
```

Output

```
int64
```

Creating Special Arrays

ZEROS

```
import numpy as np  
  
print(np.zeros(5))
```

Output

```
[0. 0. 0. 0. 0.]
```

ONES

```
print(np.ones(4))
```

Output

```
[1. 1. 1. 1.]
```

IDENTITY MATRIX

```
print(np.eye(3))
```

Output

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

ARANGE()

Like range()

```
print(np.arange(1,11))
```

Output

```
[1 2 3 4 5 6 7 8 9 10]
```

Linspace()

Creates equally spaced numbers.

```
print(np.linspace(0,10,5))
```

Output

```
[0. 2.5 5. 7.5 10.]
```

Array Indexing

```
arr = np.array([100,200,300])  
  
print(arr[0])
```

Output

```
100
```

Negative Index

```
print(arr[-1])
```

Output

```
300
```

Array Slicing

```
arr = np.array([10,20,30,40,50])
```

```
print(arr[1:4])
```

Output

```
[20 30 40]
```

Mathematical Operations

```
arr = np.array([10,20,30])
```

```
print(arr+10)
```

Output

```
[20 30 40]
```

Subtract

```
print(arr-5)
```

Output

```
[5 15 25]
```

Multiply

```
print(arr*2)
```

Output

```
[20 40 60]
```

Divide

```
print(arr/2)
```

Output

```
[5. 10. 15.]
```

Aggregate Functions


```
sales = np.array([12000,25000,18000,30000])
```

Sum

```
print(np.sum(sales))
```

Output

```
85000
```

Mean

```
print(np.mean(sales))
```

Output

```
21250
```

Maximum

```
print(np.max(sales))
```

Output

```
30000
```

Minimum

```
print(np.min(sales))
```

Output

```
12000
```

Standard Deviation

```
print(np.std(sales))
```

Reshape

Convert 1D to 2D

```
arr = np.array([1,2,3,4,5,6])

print(arr.reshape(2,3))
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Flatten

Convert back to 1D

```
arr = np.array([
    [1,2],
    [3,4]
])

print(arr.flatten())
```

Output

```
[1 2 3 4]
```

Real Data Analyst Examples

EXAMPLE 1: SALES ANALYSIS

```
import numpy as np

sales = np.array([12000,25000,18000,30000,22000])

print("Total :", np.sum(sales))
print("Average :", np.mean(sales))
print("Highest :", np.max(sales))
print("Lowest :", np.min(sales))
```

EXAMPLE 2: EMPLOYEE SALARY INCREMENT

```
import numpy as np

salary = np.array([25000,30000,40000])
```

```
new_salary = salary * 1.10

print(new_salary)
```

Output

```
[27500. 33000. 44000.]
```

EXAMPLE 3: STUDENT MARKS

```
import numpy as np

marks = np.array([75,80,95,60,85])

print("Average :", np.mean(marks))
print("Highest :", np.max(marks))
```

Mini Project 1: Salary Dashboard

```
import numpy as np

salary = np.array([30000,35000,40000,45000])

print("Total Salary :", np.sum(salary))
print("Average Salary :", np.mean(salary))
```

Mini Project 2: Profit Analysis

```
import numpy as np

profit = np.array([1000,-500,2500,-300,4500])

print("Total Profit :", np.sum(profit))
print("Maximum Profit :", np.max(profit))
```

Practice Questions

BASIC

1. Import NumPy.

2. Create a 1D array.
 3. Create a 2D array.
 4. Print array dimensions.
 5. Print array shape.
-

ARRAY CREATION

6. Create a zeros array.
 7. Create a ones array.
 8. Create an identity matrix.
 9. Create an array using `arange()`.
 10. Create an array using `linspace()`.
-

INDEXING & SLICING

11. Print the first element.
 12. Print the last element.
 13. Slice the first 3 elements.
 14. Slice the last 2 elements.
 15. Use negative indexing.
-

MATHEMATICAL OPERATIONS

16. Add 5 to every element.
 17. Multiply every element by 3.
 18. Divide every element by 2.
 19. Subtract 10 from every element.
 20. Square every element.
-

AGGREGATE FUNCTIONS

21. Find sum.
 22. Find average.
 23. Find maximum.
 24. Find minimum.
 25. Find standard deviation.
-

BUSINESS PROBLEMS

26. Monthly sales analysis.

- 27. Employee salary analysis.
 - 28. Student marks analysis.
 - 29. Profit analysis.
 - 30. Sales target comparison.
-

Assignment (Data Analyst Level)

Sales Data:

```
import numpy as np

sales = np.array([
    12000,
    25000,
    18000,
    30000,
    22000,
    27000,
    35000
])
```

Calculate:

- Total Sales
- Average Sales
- Highest Sale
- Lowest Sale
- Standard Deviation

Then increase every sale by **10%** and print the new values.

Interview Questions

BASIC

1. What is NumPy?
2. Why is NumPy faster than Python lists?
3. What is an array?
4. Difference between a list and a NumPy array?
5. How do you import NumPy?

INTERMEDIATE

6. What is `ndim` ?
7. What is `shape` ?
8. What is `dtype` ?
9. Difference between `arange()` and `linspace()` ?
10. What is `reshape()` ?

ADVANCED

11. Why is NumPy used in Data Analysis?
 12. How does vectorization improve performance?
 13. What are aggregate functions?
 14. What is broadcasting in NumPy? (*We'll cover this in Day 12.*)
 15. Why do Pandas and Machine Learning libraries depend on NumPy?
-

Learning Outcome

After **Day 11**, you will be able to:

-  Install and import NumPy
-  Create and manipulate arrays
-  Understand array dimensions, shape, size, and data types
-  Perform fast mathematical operations
-  Use aggregate functions for data analysis
-  Solve real-world business problems using NumPy arrays